



递归及其回溯算法

引例、借书问题

[问题描述]

学校放暑假时，信息学辅导教师有n本书要分给参加培训的n个学生。如A，B，C，D，E共5本书要分给参加培训的张、刘、王、李、孙5位学生，每人只能选1本。教师事先让每个人将自己喜爱的书填写在如下的表中，然后根据他们填写的表来分配书本，希望设计一个程序帮助教师求出可能的分配方案，使每个学生都满意。

	A	B	C	D	E
张			Y	Y	
王	Y	Y			
刘		Y	Y		
孙				Y	
李		Y			Y

输入格式第一行一个数 n （学生的个数，书的数量）
以下共 n 行，每行 n 个0或1（由空格隔开），第 i 行数据表示第 i 个同学对所有书的喜爱情况。0表示不喜欢该书，1表示喜欢该书。

输出格式依次输出每个学生分到的书号。

样例

输入book.in

5

0 0 1 1 0

1 1 0 0 0

0 1 1 0 0

0 0 0 1 0

0 1 0 0 1

输出book.out

3 1 2 4 5

第一行输出方案总数

以下每行输出一个可行方案、每个数字减用空格隔开
（按照字典序最小顺序输出）

思路转变

针对这个问题，发现和我们之前学习的问题都不一样，那不一样的地方在哪里呢？

这个问题不再是一句话题意了？换句话说之前讲的知识主要集中在C语言语法本身上，而题目的思维难度都非常低，基本上都是一句话题意、意思都懂关键是看用代码实现，然而我们要搞好竞赛那远远不够的。我们要学习的是用C语言解决具体实际问题，所以从现在开始我们将从语法上转变到思维上。特别是题目的思考上。所以那些如果现在为止还对前面语法不熟悉的同学，那你自己必须想办法了，必须要自己去补，否则你就只能离开信息学竞赛班了。

借书问题分析

那么针对这个问题我们该如何来分析呢？

针对这个问题，我们发现是 n 个人借 n 本书的问题，因此我们可以从每个学生出发，也可以从每本书出发。

那我们从每个学生出发思考，针对每个学生只能借一本书，但是必须是这个学生喜欢的，而且一本书也只能借给一个人。然而可以借的书呢有 n 本，所以要想每个人都要借到满意的书，所以还需要从整体上去考虑，也就是说只有每个人都借到自己满意的书才是一个方案。

5

0 1 1 1 0

1 1 1 0 0

0 1 1 1 0

0 1 0 1 1

0 1 0 0 1

针对样例输入我们分析下第1个学生只可以借第3和第4本书，第2个学生可以借第1和第二本书，那到底第1个人和第2个人借那本书才能满足我们的要求呢？这个还需要考虑后面学生喜欢书的情况而定。

借书问题总结分析

所以总体来分析针对第 i 个学生借书，我们要考虑以下3点

- 1、第 i 个学生可以借书的范围。
- 2、 i 是否喜欢这本书。
- 3、 i 喜欢的这本书是否还可以借(也就是前面 $i-1$ 个人还没有把这本书借走。)
- 4、考虑是否 n 个人都借到了自己满意的本书，如果是这样子，才表示得到一个借书方案

那么这个问题用什么算法思想来实现呢？

我们分析发现这里针对每个人思考的方式都是一样的，所以可以用我们学习的递归来实现。

针对第i个人借书代码描述

//第i个人借书，说明前i-1个人已经借了书

```
void try(int i)
```

```
{
```

```
    if (i==n+1) {输出结果; } //前n个人都借了书
```

```
    for (int j=1;j<=n;j++) //搜索人i能够可以借的书的范围
```

```
        if (第i个人可以借第j 本书)
```

```
        {
```

```
            记录下b[i]=j;
```

```
            book[j]=false;//标记第j本数已被借
```

```
            try(i+1);
```

```
            book[j]=true; //删除书的被借标志，别人可以借
```

```
        }
```

```
    }
```

针对第i个人借第j本书如果第i个人借第j本书这样子一直做下去，但是不能找到1种满足题目要求的方案，则第i个人不能借第j本书，所以要把book[j]=true,第i个人尝试借下一本书。

具体实现代码分析

```
int a[10+10][10+10]; //a[i][j]=1第i个人喜欢第j本书, 0表示不喜欢}
int b[10+10]; //记录分配方案b[i]是第i个人借第b[i]本书
bool book[10+10];
/*book[[i]能否可以借第i本书,是否他人已借此书, 初始true,一旦有人借了就
改为false*/
int n; //n本书
```

void init() //初始化函数，实现数据读入，数组初始化。

```
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
    memset(book,true,sizeof(book));
}
```

//打印数据的函数

```
void print()
{
    for(int i=1;i<n;i++)
    {
        printf("%d ",b[i]);
    }
    printf("%d\n",b[n]);
}
```

//第i个人可以借的书函数

```
void dfs(int x){
    if(x==n+1){
        print();
    }
    for(int i=1;i<=n;i++)
    {
        if(a[x][i]==1 && book[i])
        {
            book[i]=false;
            b[x]=i;
            dfs(x+1);
            book[i]=true;
        }
    }
}
```

程序主函数

```
int main()
{
    init();
    dfs(1);
    return 0;
}
```

解决这类问题的方法回溯法

回溯法

回溯法（探索与回溯法）是一种选优搜索法，又称为试探法，按选优条件向前搜索，以达到目标。但当探索到某一步时，发现原先选择并不优或达不到目标，就退回一步重新选择，这种走不通就退回再走的技术为回溯法，而满足回溯条件的某个状态的点称为“回溯点”。

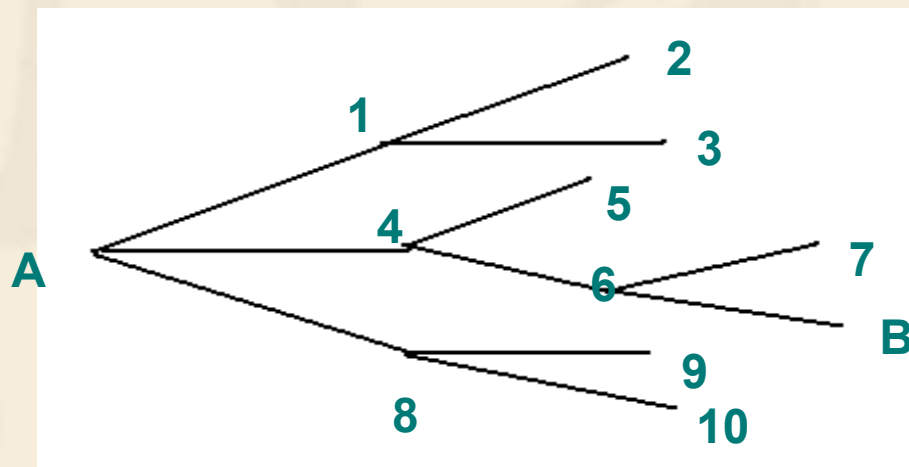
基本思想

在包含问题的所有解的解空间树中，按照深度优先搜索的策略，从根结点出发深度探索解空间树。当探索到某一结点时，要先判断该结点是否包含问题的解，如果包含，就从该结点出发继续探索下去，如果该结点不包含问题的解，则逐层向其祖先结点回溯。（其实回溯法就是对隐式图的深度优先搜索算法）。若用回溯法求问题的所有解时，要回溯到根，且根结点的所有可行的子树都要已被搜索遍才结束。而若使用回溯法求任一个解时，只要搜索到问题的一个解就可以结束。

从问题的某一种可能出发, 搜索从这种情况出发所能达到的所有可能, 当这一条路走到“尽头”而没达到目的地的时候, 再倒回上一个出发点, 从另一个可能出发, 继续搜索. 这种不断“**倒回一步**”寻找解的方法, 称作“回溯法”.

回溯即是较简单、较常用的搜索策略, 实质就是一种搜索策略.

如找一条从A到B的路线



从A到B的路线A---4---6---B

递归算法框架

```
void dfs(int i);{搜索第i个分量 $X_i$ }
```

```
{
```

```
    if (i==n +1) print; {得到一种解}
```

```
    for  $X_i \in S_i$  且使得  $(X_1, X_2, \dots, X_{i-1}, X_i)$  //枚举 $x_i$ 分量的范围
```

```
{
```

```
        if(不冲突)
```

```
{
```

```
            记录满足条件的 $X_i$ ;
```

```
            添加相应的标志;
```

```
            try(i+1)
```

```
            删除标志;恢复之前的状态,根据具体情况选择
```

```
        }
```

```
    }
```

```
}
```

例题组合数学

在1到n的n个数中选出m个数的方案数和具体方案，
(输出方案按照字典序从小到大的顺序输出)

这个题的思考思路和选书是一模一样的，唯一不一样的地方是我们从N个数中选出M个

void dfs(int k,int x)//递归求解方案函数，dfs(k,x)表示第k个位置从数x开始选数

```
{  
    if(k==m+1)//k==m表示。已经找到了m个元素，因此是一个方案，所以必须打印出来  
    {  
        print();//调用打印函数  
    }  
    else  
    {  
        for(int i=x;i<=n;i++)  
        /*理解i=x,为什么？ 为了避免重复，所以，我们只枚举从x开始及其后面的数*/  
        {  
            ac[k]=i;  
            dfs(k+1,i+1);  
        }  
    }  
}
```

```
#include<stdio>
#include<algorithm>
#include<cstring>
using namespace std;
int n,m,total=0;
int ac[10];//记录组合方案的数字
void init();//文件操作函数
{
    freopen("zuhe.in","r",stdin);
    freopen("zuhe.out","w",stdout);
    scanf("%d%d",&n,&m);
}
```

```
void print();//打印组合方案函数
{
    for(int j=1;j<m;j++)
    {
        printf("%d ",ac[j]);
    }
    printf("%d\n",ac[m]);
    total++;//记录所有方案总数
}
```

```
int main()
{
    init();
    //初始化，表示一开始每个数都是可以使用的
    dfs(1,1);//dfs(i,k)表示第i个位置从数k开始选数
    printf("%d\n",total);//输出方案总数
    //while(1);
    return 0;
}
```

一 N皇后问题

[问题描述]

在 $n \times n$ 的国际象棋棋盘上，放置 n 个皇后，使任何一个皇后都不能吃掉另一个，要使任何一个皇后都不能吃掉另一个，需满足的条件是同一行、同一列、同一对角线上只能有一个皇后。求放置方法。

如 $n=4$ 时,有以下2种放置方法.

	*		
			*
*			
		*	

		*	
*			
			*
	*		

输出

2 4 1 3

3 1 4 2

方法一分析

1、问题解的形式

`int x [8+10];`

{`x[i]`第*i*个皇后放在第*i*行，第`x[i]`列，保证所有皇后不同行}

问题的解变成求 (`x[1]`, `x[2]`, ... `x[n]`) ; `x[i] ∈ {1,2,3,4}`

4皇后问题的解 (2, 4, 1, 3) , (3, 1, 4, 2)

	*		
			*
*			
		*	

		*	
*			
			*
	*		

2、放置第 k ($1 \leq k \leq n$) 个皇后的递归算法

```
void dfs(int k){
```

```
//搜索第 $k$ 个皇后所在的列 $x[k]=?$  ,前 $k-1$ 个已放好,即已求得 $x[1]...x[k-1]$ 
```

```
if (k==n+1) print (输出放置方案数组 $x$ ) ;
```

```
for(int i=1;i<=n;i++) //搜索第 $k$ 个皇后所在的列 $i$ 
```

```
if (第 $k$ 个皇后能够放置在第 $i$ 列 )
```

```
{
```

```
    放置第 $k$ 个皇后在第 $i$ 列 ( $x[k]=i$ ) ;
```

```
    dfs(k+1);
```

```
}
```

```
}
```

3、怎样判断第k个皇后能否放置在第i列

```
bool place(int k,int i) //第k个皇后能否放在第i列
{
    for (int j=1;j<=k-1;j++) //枚举前k-1个皇后时候冲突
    {
        if((x[j]==i) || (abs(x[j]-i)=abs(j-k)))
        {
            return false;
        }
    }
    return true;
}
```

方法二

方法一的缺点每次调用函数**place** (**i,j**) 判断第**i**个皇后能否放在**j**列时有一个**for**循环判断，显然浪费时间。

```
bool place(int k,int i)
```

```
{第k个皇后能否放在第i列}
```

```
{
```

```
    for (int j=1;j<=k-1;j++)
```

```
    {
```

```
        if (x[j]==i) || (abs(x[j]-i)=abs(j-k))
```

```
        {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

int x[100]; // x[i]第i个皇后放在第i行，第x[i]列，保证所有皇后不同行
bool a[100]; //列控制标志false可以放，true不能放
bool b[100]; //左上右下方斜线控制标志，false可以放，true不能放
bool c[100]; //左下右上方斜线控制标志，false可以放，true不能放
//数据初始化

```
memset(x,sizeof(x),0);  
memset(a,sizeof(a),0);  
memset(b,sizeof(b),0);  
memset(c,sizeof(c),0);
```

递归算法

```
void dfs (int i)
```

```
{
```

```
    if (i==n+1) print
```

```
    else
```

```
    {
```

```
        for(int j=1;j<=n;j++)
```

```
        {
```

```
            if(!a[j] && !b[i+j] && !c[i-j+10])
```

```
            {
```

```
                x[i] =j;
```

```
                a[j] =true;   {列控制标志}
```

```
                b[i+j] =true; {左上右下方斜线控制标志}
```

```
                c[i-j+10] = true; {左下右上方斜线控制标志}
```

```
                dfs(i+1);
```

```
{如果不能递归进行，既a[j] and b[i+j] and c[i-j+10]=false
```

```
无法放置i+1个皇后，说明当前皇后i放置不正确，要回溯，消除标志}
```

```
                a[j]=false;
```

```
                b[i+j] =false;
```

```
                c[i-j+10] =false;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

2、无重复元素的全排列

输入 n ($n \leq 10$ 并且输入的数字在0~100之间) 个不同的数字，输出 n 个数字的全部排列。

样例输入

3

1 23 45

样例输出

11 23 45

21 45 23

323 1 45

423 45 1

545 1 23

645 23 1

问题的分析这个问题同我们前面讲的组合问题是十分一致的，唯一不同的是这里选出的数必须还有顺序问题。那怎么来处理才会导致方案不出现重复了？这里发现完全可以不去看这个时候会出现重复，因为只要数不出现重复，则不会不出现重复，我们只需要考虑，让这个数在一个方案中出现一次就可以了。

所以问题的求解思路非常简单，直接枚举每个位置可以取的数的范围，同时满足这个数在前面是没有使用过的就可以了

void dfs(int k)//第k位置取数

{

if(k==n+1)

{

print();

}

else

{

for(int i=1;i<=n;i++)//第i位置可以取数的范围从第一个数到第n个数

{

if(visit[number[i]])

{

visit[number[i]]=false;

f[k]=number[i];

dfs(k+1);

visit[number[i]]=true;

}

}

}

}

```
#include<stdio>
#include<cstring>
#include<algorithm>
using namespace std;
bool visit[100+10]; //visit[i]判断i时候使用过
int number[10+10]; //输入的原始数
int f[10+10]; //记录方案
int n;
int total=0;
void init()
{
    memset(visit,true,sizeof(visit));
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&number[i]);
    }
}
```

```
void print()
{
    bool first=true;
    for(int i=1;i<=n;i++)
    {
        if(first)
        {
            printf("%d",f[i]);
            first=false;
        }else
            printf(" %d",f[i]);
    }
    printf("\n");
    total++;
}
```

4、自然数n的分解

分解（一）

输入自然数n（ $n < 100$ ），输出所有和的形式。不能重复。

如 $4=1+1+2$ ； $4=1+2+1$ ； $4=2+1+1$ 属于一种分解形式。

样例输入

7

输出

$$17=1+6$$

$$27=1+1+5$$

$$37=1+1+1+4$$

$$47=1+1+1+1+3$$

$$57=1+1+1+1+1+2$$

$$67=1+1+1+1+1+1+1$$

$$77=1+1+1+2+2$$

$$87=1+1+2+3$$

$$97=1+2+4$$

$$107=1+2+2+2$$

$$117=1+3+3$$

$$127=2+5$$

$$137=2+2+3$$

$$147=3+4$$

```
int n,k,i,count;
int a[100];//记录分解项
void print(int x) //输出分解方案
{
    count++;
    printf("%d%d=",count,n);
    for (int i=1;i<=k;i++) printf("%d+",a[i]);
    printf("\n") {输出分解方案}
}
void try(int x) //要分解x，前面已有k项
{
    print(x);
    for (int j=a[k];j<=x/2;j++) {分解x=j + (x-j) 保证非递减}
    {
        if (j>=a[k])&&(x-j>=j)
        {
            k++;
            a[k] =j;
            try(x-j);
            k--;
        }
    }
}
```

```
int main(){  
    scanf("%d",&n);  
    count=0;  
    for (int i=1;i<=n/2;i++){先分解成i+ (n-i) 两项和}  
    {  
        k=1;  
        a[1] =i;  
        try(n-i);  
    }  
}
```

分解（二）

输入自然数n和m（n，m<100），输出所有分解，分解后的每一项都不超大于m的。不能重复。如4=1+1+2；4=1+2+1；4=2+1+1 属于一种分解形式。

如

输入

7

4

输出

$$17=1+1+1+4$$

$$27=1+1+1+1+3$$

$$37=1+1+1+1+1+2$$

$$47=1+1+1+1+1+1+1$$

$$57=1+1+1+2+2$$

$$67=1+1+2+3$$

$$77=1+2+4$$

$$87=1+2+2+2$$

$$97=1+3+3$$

$$107=2+2+3$$

$$117=3+4$$

```
var n,m,k,i,countinteger;  
  aarray[1..100] of integer;  
procedure print(i,xinteger);  
  var jinteger;  
  begin  
    inc(count);  
    write(count,"n, '=');  
    for i=1 to k do write(a[i], '+');  
    writeln(x);  
  end;  
procedure try(xinteger);{x是第k+1项}  
  var i,jinteger;  
  begin  
    if x<=m then print(k,x);  
    for j=a[k] to x div 2 do  
      if (j>=a[k])and(x-j>=j) then  
        begin  
          inc(k);  
          a[k]=j;  
          try(x-j);  
          dec(k);  
        end;  
      end;  
    end;  
end;
```

```
begin  
  readln(n);  
  readln(m);  
  count=0;  
  for i=1 to n div 2 do  
    begin  
      k=1;  
      a[1]=i;  
      try(n-i);  
    end;  
  end.  
end.
```

分解（三）

输入自然数 n 和 m （ $n, m < 100$ ），输出所有分解项数不超过 m 的所有形式。
不能重复。如 $4=1+1+2$ ； $4=1+2+1$ ； $4=2+1+1$ 属于一种分解形式。

如

输入

7 4

输出

$17=1+6$

$27=1+1+5$

$37=1+1+1+4$

$47=1+1+2+3$

$57=1+2+4$

$67=1+2+2+2$

$77=1+3+3$

$87=2+5$

$97=2+2+3$

$107=3+4$

```

var n,m,k,i,count integer;
  aarray[1..100] of integer;
procedure print(i,x integer);
  var j integer;
  begin
    inc(count);
    write(count,"n, '=');
    for i=1 to k do write(a[i], '+'); writeln(x);
  end;
procedure try(x integer); {x是第k+1项}
  var i,j integer;
  begin
    if k <= m-1 then print(k,x);
    if k >= m then exit; {剪枝优化}
    for j=a[k] to x div 2 do
      if (j >= a[k]) and (x-j >= j) then
        begin
          inc(k);
          a[k]=j;
          try(x-j);
          dec(k);
        end;
    end;
  end;
end;

```

```

begin
  readln(n);
  readln(m);
  count=0;
  for i=1 to n div 2 do
    begin
      k=1;
      a[1]=i;
      try(n-i);
    end;
  end.

```

算法的缺点？

改进优化？

7、细胞

一矩形阵列由数字0到9组成, 数字1到9代表细胞, 细胞的定义为沿细胞数字上下左右还是细胞数字则为同一细胞, 求给定矩形阵列的细胞个数。

输入整数m, n (m行, n列)

矩阵

输出细胞的个数。

样例

输入

```
4 10
0234500067
1034560500
2045600671
0000000089
```

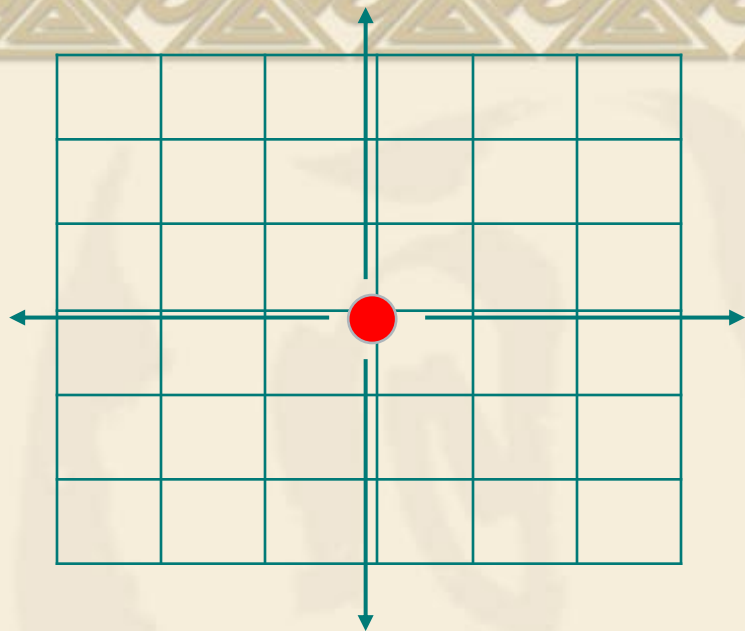
输出4

0 2 3 4 5 0 0 0 6 7

1 0 3 4 5 6 0 5 0 0

2 0 4 5 6 0 0 6 7 1

0 0 0 0 0 0 0 0 8 9



$dx[] = \{1, 0, -1, 0\}$
 $dy[] = \{0, 1, 0, -1\}$

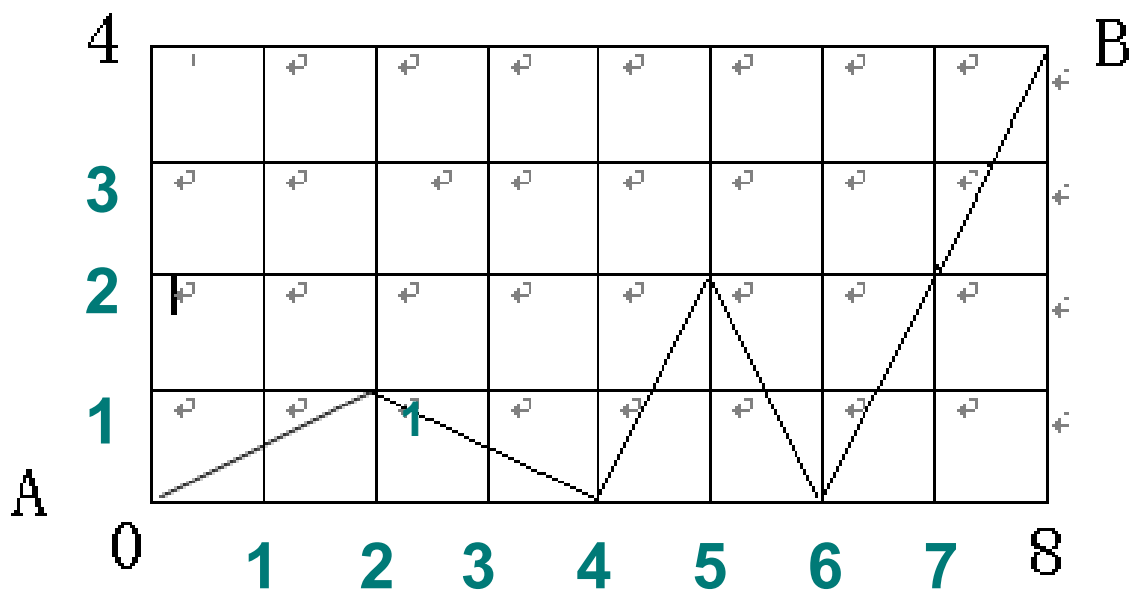
5、骑士的游历

设有下图所示的一个棋盘，在棋盘上的**A(0,0)**点有一个中国象棋马，并约定马走的规则

1、马只向右走；

2、马走“日”字。

找出所有从**A**到**B**的路径



输入**B**的坐标**n**，**m** (≤ 15)。

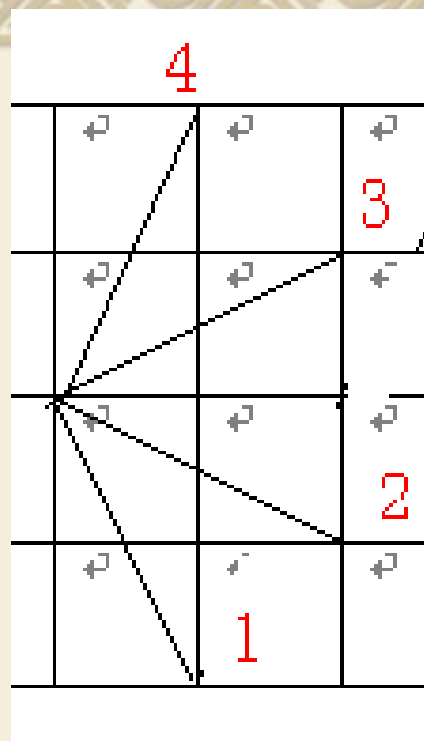
1

输出所有走法。

如

输入8 4

输出 (0,0)(2 1)(4 0)(5 2)(6 0)(7 2)(8 4)



6、迷宫问题

设有一个 $N*N$ 方格的迷宫，入口和出口分别在左上角和右上角。迷宫格子中分别放有0和1，0表示可通，1表示不能，迷宫走的规则如下图所示即从某点开始，有八个方向可走，前进方格中数字为0时表示可通过，为1时表示不可通过，要另找路径。

输入例子（从文件中读取数据）

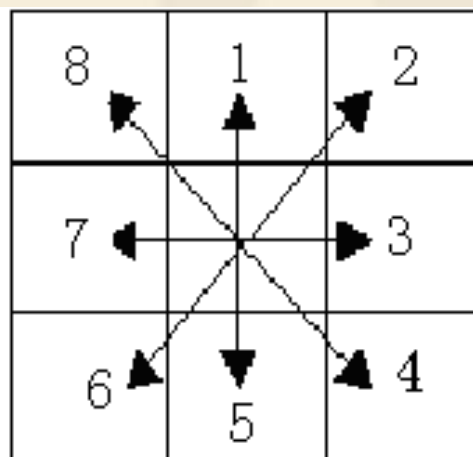
8

```
0 0 0 1 1 0 1 0
1 0 1 1 0 1 1 0
0 1 0 0 1 0 0 1
0 0 1 1 0 1 0 1
0 1 0 0 0 1 1 0
0 1 1 1 1 1 0 1
0 0 1 1 1 0 1 1
1 1 0 0 0 0 0 0
```

入口（1，1）；出口（1，8）

输出要求找出一条 从入口（左上角）到出口（又上角）的路径(不能重复)。

(1,1)->(2,2)->(3,3)->(3,4)->(4,5)->(3,6)->(3,7)->(2,8)->(1,8)



8个方向的位移

$dx[1]=0; \quad dy[1]=-1;$

$dx[2]=1; \quad dy[2]=-1;$

$dx[3]=1; \quad dy[3]=0;$

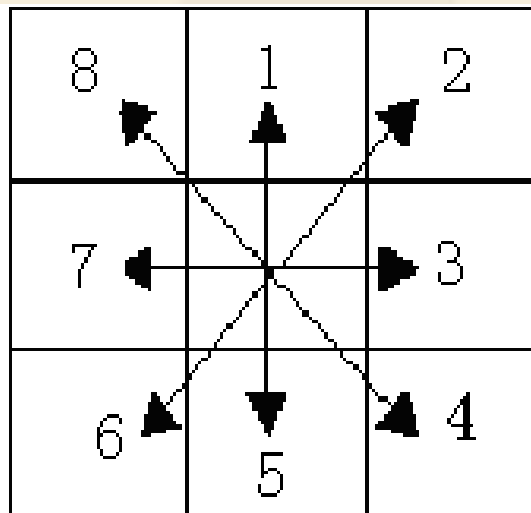
$dx[4]=1; \quad dy[4]=1;$

$dx[5]=0; \quad dy[5]=1;$

$dx[6]=-1; \quad dy[6]=1;$

$dx[7]=-1; \quad dy[7]=0;$

$dx[8]=-1; \quad dy[8]=-1;$



8、组合

(一) 无重复元素的组合

输入一串小些字母（无重复字母），从中取出k个字母，输出组合情况。

样例

输入

abcd

3

输出

abc

abd

acd

bcd

（二）、有重复元素的组合

输入一串小些字母（有重复字母），从中取出k个字母，输出组合情况。

样例

输入

aabbcc

4

输出

1aabb

2aabc

3aacc

4abbc

5abcc

6bbcc

9、数字排列(li3.txt)

在一个 $N*N$ 的棋盘上 ($1 \leq n \leq 10$), 填入 $1, 2, \dots, n*n$ 共 $n*n$ 个数, 使得任意两个相邻的数之合为素数

例如

$n=2$ 时, 有

1 2

4 3

$n=4$

1 2 11 12

16 15 8 5

13 4 9 14

6 7 10 3

10、因式分解

输入自然数 n ($<10^9$), 将 n 分解成一系列自然数乘积的形式

$N=a_1*a_2*.....*a_m$, $1<a_1\leq a_2\leq.....\leq a_m<n$.

输出所有的分解方案及方案数目。

样例

输入

12

输出

2*6

2*2*3

3*4

3

2、无重复元素的全排列

输入n (≤ 10) 个不同的小些字母，输出n个字符的全部排列。

样例

输入

abc

输出

1abc

2acb

3bac

4bca

5cab

6cba

3、有重复元素的全排列

输入n（ ≤ 10 ）个小些字母（可能重复），输出n个字符的全部排列。

样例

输入

abaab

输出

1aaabb

2aabab

3aabba

4abaab

5ababa

6abbba

7baaab

8baaba

9babaa

10bbaaa